

Statistics 6910: XML (eXtensible Markup Language)

Most of the data sets we have seen have been in the form of ASCII tables.

Date	Time	Lat	Lon	Depth	Mag
1968/01/12	22:19:10.35	36.6453	-121.2497	6.84	3.00
1968/02/09	13:42:37.05	37.1527	-121.5448	8.49	3.00
1968/02/21	14:39:48.10	37.1783	-121.5780	6.95	3.80
1968/03/02	04:25:53.94	36.8343	-121.5447	5.35	3.00
1968/03/17	15:07:02.12	37.3088	-121.6615	4.39	3.00
1968/03/21	21:54:59.94	37.0378	-121.7407	11.86	4.30

- Advantages:
 - easy to read, write, and process
 - in standard cases, don't need a lot of extra information
- But these advantages can quickly disappear....

XML is a standard for *semantic, hierarchical* representation of data

```
<state>
<gml:name abbreviation="AL"> ALABAMA </gml:name>
<county>
<gml:name> Autauga County </gml:name>
<gml:location>
<gml:coord>
<gml:X> -86641472</gml:X>
<gml:Y> 32542207</gml:Y>
</gml:coord>
</gml:location>
</county>
```

Relationships between pieces of data reflect relationships in the real world.

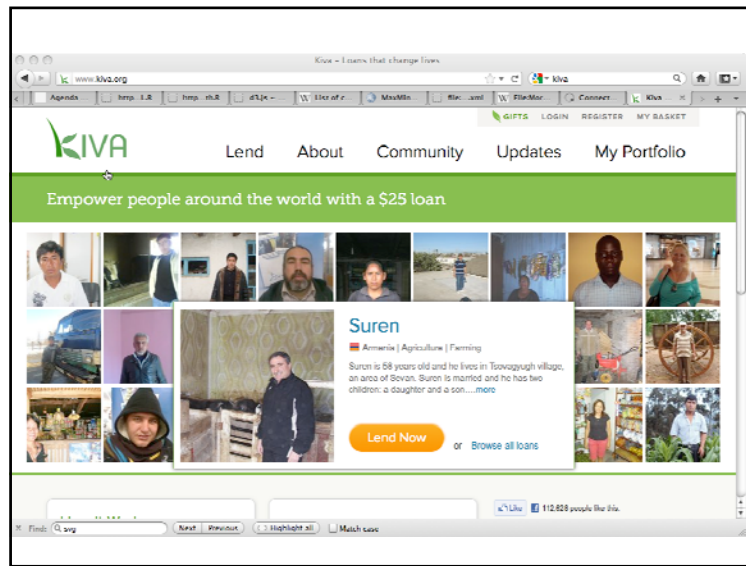
Pros

- data is self-describing
- format separates content from structure
- data can be easily merged and exchanged
- file is human-readable
- file is also easily machine-generated
- standards are widely adopted

Cons

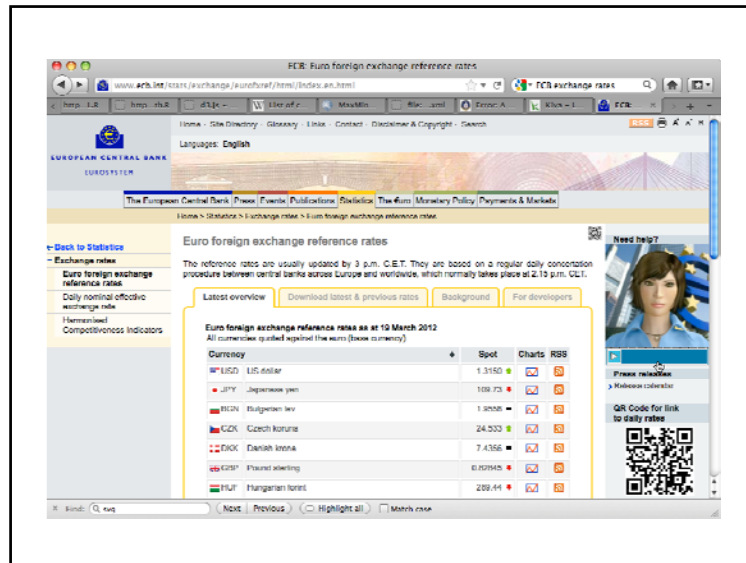
- XML documents can be very verbose and hard to read
- It's so general that it's hard to develop tools for all cases
- Files can be quite large due to high amount of redundancy

- XML has become quite popular in many scientific fields, and it is standard in many web applications for the exchange and visualization of data.
- We'll learn how to
 - create it and
 - read/process it.
- We'll do both of these things from within R, but first let's start with an overview of what XML documents look like.



Snippet of Kiva Data for one lender

```
<lender>
  <lender_id>matt</lender_id>
  <name>Matt</name>
  <image>
    <id>12829</id>
    <template_id>1</template_id>
  </image>
  <whereabouts>San Francisco CA</whereabouts>
  <country_code>US</country_code>
  <uid>matt</uid>
  <member_since>2006-01-01T09:01:01Z</member_since>
  <personal_url>www.socialedge.org/blogs/kiva-chronicles</personal_url>
  <occupation>Entrepreneur</occupation>
  <loan_because>I love the stories. </loan_because>
  <occupational_info>I co-founded a startup nonprofit (this one!)
    and I work with an amazing group of people dreaming up ways to
    alleviate poverty through personal lending.
  </occupational_info>
  <loan_count>89</loan_count>
  <invitee_count>23</invitee_count>
</lender>
```



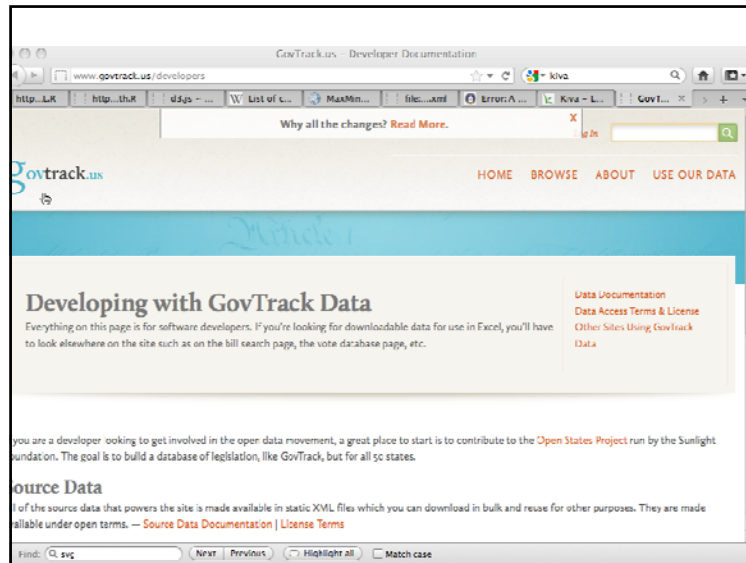
Snippet of exchange data

```
<Cube>
<Cube time="2008-04-21">
<Cube currency="USD" rate="1.5898"/>
<Cube currency="JPY" rate="164.43"/>
<Cube currency="BGN" rate="1.9558"/>
<Cube currency="CZK" rate="25.091"/>
</Cube>
<Cube time="2008-04-17">
<Cube currency="USD" rate="1.5872"/>
<Cube currency="JPY" rate="162.74"/>
<Cube currency="BGN" rate="1.9558"/>
<Cube currency="CZK" rate="24.975"/>
</Cube>
```



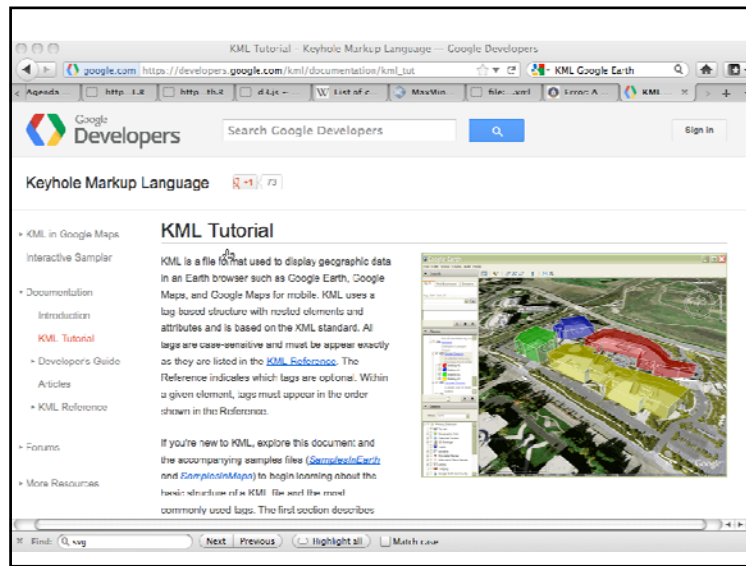
Snippet of USGS earthquake catalog data

```
<event id="00068404" network-code="ak"
time-stamp="2008/09/16_22:17:31" version="2">
<param name="year" value="2008"/>
<param name="month" value="09"/>
<param name="day" value="14"/>
<param name="hour" value="00"/>
<param name="minute" value="59"/>
<param name="second" value="04.0"/>
<param name="latitude" value="51.8106"/>
<param name="longitude" value="-175.9250"/>
<param name="depth" value="146.0"/>
<param name="magnitude" value="3.8"/>
<param name="num-stations" value="10"/>
<param name="num-phases" value="15"/>
<param name="dist-first-station" value="126.1"/>
<param name="azimuthal-gap" value="53"/>
<param name="magnitude-type" value="L"/>
<param name="magnitude-type-ext"
value="MI = local magnitude (synthetic Wood-Anderson)"/>
<param name="location-method" value="a"/>
<param name="location-method-ext"
value="Auryn (Confirmed by human review)"/>
</event>
<event>
```



```
<actions>
<action datetime="2009-01-26">
  <text>Referred to the Committee on Appropriations, and in addition
    to the Committee on the Budget, for a period to be
    subsequently determined by the Speaker, in each case for
    consideration of such provisions as fall within the
    jurisdiction of the committee concerned.
  </text>
</action>
<action datetime="2009-01-26">
  <text>Referred to House Appropriations</text>
</action>
</actions>
..
<relatedbills>
  <bill relation="rule" session="111" type="hr" number="88" />
</relatedbills>
```

Snippet of
US
Congress
data



```
<Placemark id="217">
  <name>8.2</name>
  <description>
    Date: 2008-9-15
    Magnitude: 1.5
    Depth: 8.2 km
  </description>
  <styleUrl>#ball1-2</styleUrl>
  <Point>
    <coordinates>-147.426, 60.929, 0</coordinates>
  </Point>
</Placemark>
```

Snippet of
KML for one
earthquake

XML Syntax

Syntax

The basic unit of XML code is called an “element” or “node.” It is made up of both *markup* and content. Markup consists of *tags*, *attributes*, and *comments*.

```
<CYL> 6 </CYL> <!-- elem with content 6 -->
```

Start tag Content End tag Comment - can go anywhere

```
<CYL> </CYL>
```

Elements with no content

```
<CYL type="numeric" />
```

An attribute

```
<CYL size="2"> 6 </CYL>
```

Well-formed

- Tag names are case-sensitive; start and end tags must match exactly.
- No spaces are allowed between the `<` and the tag name.
- Tag names must begin with a letter and contain only alphanumeric characters.
- An element must have both an open and closing tag unless it is empty.
- An empty element that does not have a closing tag must be of the form `<tagname/>`.
- Tags must nest properly. (Inner tags must close before outer ones.)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!-- Edited with XML Spy v2006 (http://www.astova.com) -->
<CATALOG>
  <PLANT>
    <COMMON>Bloodroot</COMMON>
    <BOTANICAL>Sanguinaria canadensis</BOTANICAL>
    <ZONE>4</ZONE>
    <LIGHT>Mostly Shady</LIGHT>
    <PRICE>$2.44</PRICE>
    <AVAILABILITY>031599</AVAILABILITY>
  </PLANT>
  <PLANT>
    <COMMON>Columbine</COMMON>
    <BOTANICAL>Aquilegia canadensis</BOTANICAL>
    <ZONE>3</ZONE>
    <LIGHT>Mostly Shady</LIGHT>
    <PRICE>$9.37</PRICE>
    <AVAILABILITY>030699</AVAILABILITY>
  </PLANT>
</CATALOG>
```

XML declaration and processing instructions

Note how indentation makes it easier to check that the tags are correctly nested.

Well formed XML ctd.:

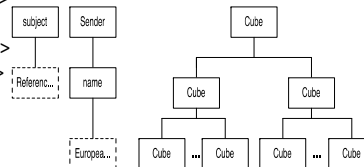
- All attributes must appear in quotes in the format:
name = "value"
- Isolated markup characters must be specified via entity references. < is specified by <; and > is specified by >.
- All XML documents must contain a *root node* containing all the other nodes.

Tree Representation

```

<Envelope>
  <subject>Reference rates</subject>
  <Sender>
    <name>European Central Bank</name>
  </Sender>
  <Cube>
    <Cube time="2008-04-21">
      <Cube currency="USD" rate="1.5898"/>
      <Cube currency="JPY" rate="164.43"/>
      <Cube currency="BGN" rate="1.9558"/>
      <Cube currency="CZK" rate="25.091"/>
    </Cube>
    <Cube time="2008-04-17">
      <Cube currency="USD" rate="1.5872"/>
      <Cube currency="JPY" rate="162.74"/>
      <Cube currency="BGN" rate="1.9558"/>
      <Cube currency="CZK" rate="24.975"/>
    </Cube>
  </Cube>
</Envelope>

```



Tree terminology

- There is only one *root or document node* in the tree, and all the other nodes are contained within it.
- We think of these other nodes as being *descendants* of the root node.
- We use the language of a family tree to refer to relationships between nodes. *Parents, children, siblings, ancestors, descendants*
- The *terminal nodes* in a tree are also known as *leaf nodes*. Content always falls in a leaf node.

Note:

- We'll learn to *create* and *process* XML documents from within R, but always keep in mind that R and XML are two separate things.
- In particular, it will be helpful to have in your mind the structure of the XML document *before* you do anything in R, especially when you're creating a new XML document.

XML package in R

- Handy functions for parsing XML
 - readHTMLTable: reads an HTML table into R
 - xmlParse: read an XML file into R
 - xmlValue: retrieve text content of a node
 - xmlSize: return the number of child nodes of a node
 - xmlSApply: applies the function to each child node of a node

- To read an XML file into R, use `xmlParse`

```
> doc = xmlParse("plant.xml")
```

and extract the root node using `xmlRoot`.

```
> root = xmlRoot(doc)
> class(root)
[1] "XMLNode"
```

To illustrate how we manipulate an XML object in R, we'll take this data and reformat it into a data frame with one row for each plant.

`xmlParse` implements what is called the DOM (Document Object Model) parser. It reads the entire file into memory.

We don't have time to cover it, but you should be aware of another parsing model called SAX (Simple API for XML). It reads the document incrementally and is more memory efficient, but it is trickier to use.

The tree structure is represented in R as a list of lists.

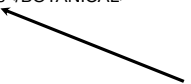
We can access an element within a node (i.e., a child), using the usual `[[ ]]` indexing for lists.

```

> ## Look at the first plant node
> oneplant = root[[1]]
> class(oneplant)
[1] "XMLNode"
> print(oneplant)
<PLANT>
<COMMON>Bloodroot</COMMON>
<BOTANICAL>Sanguinaria canadensis</BOTANICAL>
<ZONE>4</ZONE>
<LIGHT>Mostly Shady</LIGHT>
<PRICE>$2.44</PRICE>
<AVAILABILITY>031599</AVAILABILITY>
</PLANT>

```

Here is a leaf node.
It contains the content.



- We can drill down further into the list:

```

> oneplant[['COMMON']]
<COMMON>Bloodroot</COMMON>

```

- Note that this doesn't remove the markup, though. To do this, use the function `xmlValue`

```

> xmlValue(oneplant[['COMMON']])
[1] "Bloodroot"
> xmlValue(oneplant[['BOTANICAL']])
[1] "Sanguinaria canadensis"

```

- There are special XML versions of `lapply` and `sapply`, named `xmlApply` and `xmlSApply`. Each takes an XMLNode object as its primary argument. They iterate over the node's children nodes, invoking the given function.
- Like `lapply`, `xmlApply` returns a list. Like `sapply`, `xmlSApply` returns a simpler data structure if possible.

To create the data frame:

- 1) What do you want to operate on (iterate over)?
- 2) What do you want to produce?

In our plants example, we can use `xmlSApply` to extract the common names of all the plants.

```
> commons = xmlSApply(root, function(x){
  xmlValue(x[["COMMON"]])})
> head(commons)
```

PLANT	PLANT	PLANT
"Bloodroot"	"Columbine"	"Marsh Marigold"
PLANT	PLANT	PLANT
"Cowslip"	"Dutchman's-Breeches"	"Ginger, Wild"

The elements of the root node are all plant nodes, like oneplant.

XPath

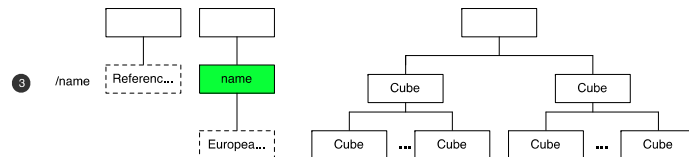
What is XPath

- Extraction tool designed for locating content in an XML file
- Uses the hierarchy of a well-formed XML document to specify the desired chunks to extract
- Syntax is similar to, but more powerful than the way files are located in a hierarchy of directories in a computer file system

```
<Envelope>
<subject>Reference rates</subject>
<Sender>
  <name>European Central Bank</name>
</Sender>
<Cube>
  <Cube time="2008-04-21">
    <Cube currency="USD" rate="1.5898"/>
    <Cube currency="JPY" rate="164.43"/>
    <Cube currency="BGN" rate="1.9558"/>
    <Cube currency="CZK" rate="25.091"/>
  </Cube>
  <Cube time="2008-04-17">
    <Cube currency="USD" rate="1.5872"/>
    <Cube currency="JPY" rate="162.74"/>
    <Cube currency="BGN" rate="1.9558"/>
    <Cube currency="CZK" rate="24.975"/>
  </Cube>
</Cube>
</Envelope>
```

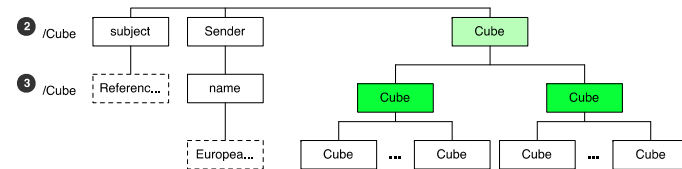
XPath Expression:
`/Envelope/Sender/name`

/Envelope/Sender/name



1. The first location step identifies the root node, **<Envelope>**.
2. The next location step finds the **<Sender>** child of **<Envelope>**.
3. The third location step identifies **<Sender>**'s child called **<name>**.

/Envelope/Cube/Cube

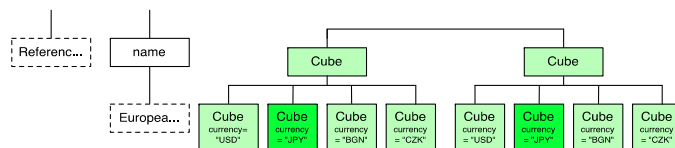


This XPath expression
locates two sibling
<Cube> nodes.

//Cube[@currency = "JPY"]

The expression: `//Cube`
matches all Cube nodes
found anywhere in the
document

The **predicate**
`[@currency="JPY"]`
filters the matches
to those elements
with a **currency**



XPath syntax

- XPath locates sets of nodes in XML files.
- An XPath expression is a *location path* that is made up of *location steps*, each of which has three parts: the axis, nodetest, and predicate, that follow
- Syntax:
`axis::nodetest[predicate]`
- The location step can be thought of as directions from one location (or context) to another.
- The nodetest is typically a node name that you wish to locate.
- The predicate filters the qualifying nodes.

XPath syntax

- We will use only very simple XPath expressions
- The axis will be either “child”, which is the default and can be dropped, or
- “descendant-or-self”, which says look anywhere down the tree and is abbreviated by “//”
- The nodetest will always be a node name
- The predicate will either be a number, [2] which asks for, e.g., the second node, or an attribute filter, e.g.,

```
[@currency = "JPY" or
 @currency="USD"]
```

Functions that take XPath expressions

- `getNodeSet(xmlTree, xpathExpression)` returns a list of XML nodes from `xmlTree` that satisfy the XPath expression.
- `xpathSApply(xmlTree, xpath, function)` the function is applied to those nodes in the XML tree that satisfy the Xpath expression. The return value is a vector when possible. `xpathApply` returns a list.

Example

```
> xpathSApply(catalog,
  "/CATALOG/PLANT/BOTANICAL", xmlValue)
[1] "Sanguinaria canadensis" "Aquilegia
canadensis" "Caltha palustris"
```

Example

```
> pieces = names(catalog[[1]])
> sapply(pieces,
  function(var) xpathSApply(catalog,
    paste("/CATALOG/PLANT/", var, sep =
  " "),
    xmlValue) )
```

COMMON	BOTANICAL	ZONE	LIGHT	PRICE
[1,] "Bloodroot"	"Sanguinaria canadensis"	"4"	"Mostly Shady"	"\$2.44"
[2,] "Columbine"	"Aquilegia canadensis"	"3"	"Mostly Shady"	"\$9.37"
[3,] "Marsh Marigold"	"Caltha palustris"	"4"	"Mostly Sunny"	"\$6.81"

```
AVAILABILITY
[1,] "031599"
[2,] "030699"
[3,] "051799"
```

One more function in XML package

```
readHTMLTable( doc,
  header = TRUE,
  colClasses = NULL,
  skip.rows = 3,
  which = 2)
```

`doc`: can be a URL, zipped file, plain text
`colClasses`: specify class to convert the values
`which`: read second table in file (when not present all tables are read)

Generating XML

XML package in R

Handy functions for creating XML

- `newXMLDoc`: create a new XML document
- `newXMLNode`: create a new XML Node
- `saveXML`: save the XML tree in a text file
- `parseXMLAndAdd`: parses character string of XML into an XML node and adds it to XMLInternalDocument (i.e. a tree)

Simple Example

```
doc = newXMLDoc()
root = newXMLNode("toplevel", doc = doc)
child1 = newXMLNode("level1", parent = root)
newXMLNode("level2", "This is the content", parent = child1)
saveXML(doc, file = "simple.xml")
```

Note: We only need to store (assign to a variable) nodes that we want to refer to as parents. For the leaf nodes, I just use `newXMLNode` without assigning. The names of the nodes in R (e.g. `root`, `child1`) are *not* part of the resulting XML file.